Much cleaner.
Much more professional.
No permission headaches.

Run that and tell me the output.

whats the difference in all these commands

Excellent question 👋
Understanding this clearly will remove 90% of your confusion.

There are **two completely different systems involved**:

- 🖥️ Linux users (OS-level)
- 🗄️ PostgreSQL users (DB-level)

You were mixing them — very common mistake.

🔹 1️⃣ `sudo -iu postgres`

👉 Switches **Linux user** to `postgres`.

What happens:

- You become the OS user `postgres`
- Your working directory becomes `/var/lib/postgresql`
- Your home directory changes
- File permissions change

Then when you run:

```
psql
```

It connects using:

- Linux user = postgres
- DB user = postgres (by default)

⚠️ Problem:
Now the Linux user `postgres` cannot access your `/home/samyak/...` project folder.

That caused your permission error.

🔹 2️⃣ `sudo -u postgres psql`

👉 Runs one command as Linux user **`postgres`**

Effect:

- Temporarily switches Linux user
- Runs **`psql`**
- Still has permission issues for your project folder

Better than **`-iu`**, but still switches OS user.

🔹 3️⃣ `psql -U postgres`

👉 DOES NOT change Linux user.

This only tells PostgreSQL:

> "Connect to DB as postgres user"

Linux user remains:

```
samyak
```

Database user becomes:

```
postgres
```

This is what you want.

✔️ Keeps file permissions intact
✔️ No directory access issues
✔️ Professional workflow

🔥 Big Difference Summary

Command	Changes Linux User?	Changes DB User?	Causes File Permission Issues?
`sudo -iu postgres`	✅ Yes	Implicitly yes	✅ Very likely
`sudo -u postgres psql`	✅ Yes	Yes	✅ Likely
`psql -U postgres`	❌ No	✅ Yes	❌ No

🎯 Correct Modern Workflow For You

Inside VS Code:

```
psql -U postgres -d corporate_db -f database/07_audit_logging.sql
```

You stay Linux user **`samyak`**.
You connect DB as **`postgres`**.

Clean separation.